S2S-07: Step 7 — Expand: OpenPipeline, SLOs, and Alerting

> **Series:** S2S — SaaS to SaaS Migration | **Notebook:** 7 of 9 | **Phase:** Run | **Step:** Expand | **Created:** March 2026 | **Last Updated:** 10/24/2026

Overview

With agents reporting to the target tenant (Step 5) and cloud integrations reconnected (Step 6), this step expands monitoring coverage to include data processing pipelines, service-level objectives, and alerting. OpenPipeline rules control how data is enriched, routed, and masked. SLOs define reliability targets. Alerting ensures the right people are notified when those targets are at risk. All three must be migrated in a coordinated sequence — buckets first, then pipelines, then SLOs and alerts that consume the processed data.

> **S2S Migration Journey — 3 Phases / 9 Steps**
>
> **Plan:** 1. Discover | 2. Strategize | 3. Design
>
> **Upgrade:** 4. Prepare | 5. Execute | 6. Integrate
>
> **Run: 7. Expand** | 8. Enable | 9. Optimize

Table of Contents

1. OpenPipeline Rule Migration
2. Grail Bucket Configuration
3. SLO Migration
4. Alerting and Notification Migration

5. Data Retention Optimization
6. Step Completion Checklist

Prerequisites

Requirement	Details
Step 5 Complete	OneAgents and DynaKube operators redirected to target tenant and reporting data
Step 6 Complete	Cloud integrations (AWS, Azure, GCP) reconnected in target tenant
Source Tenant Access	`settings.read` (API token) or `settings:objects:read` (platform token, OAuth) — OpenPipeline pipelines, ingest sources and routing are Settings 2.0 objects. To export pipelines and ingest sources owned by other users, `settings:objects:admin` (see §1)
Target Tenant Access	`settings.write` (API token) or `settings:objects:write` (platform token, OAuth); routing writes also need `settings:objects:admin`, or a policy granting `settings:objects:write` on `builtin:openpipeline.<scope>.routing`; for SLOs, an OAuth client with `slo:slos:read` / `slo:slos:write` (modern SLO app) — classic `slo.write` only if the target still runs classic SLOs
Monaco CLI	v2.x installed for bulk export/import of OpenPipeline and SLO configuration
Terraform	v1.5+ with Dynatrace provider (for IAM-gated SLO policies only)

> Older guides list `openpipeline.read` / `openpipeline.write` or `openpipeline:configurations:read` / `:write`. The first pair does not exist. The second belonged to the OpenPipeline Configurations API, which reached end of life on June 29, 2026; the Settings permissions above replace it.
>
> **Sources:** Migrate OpenPipeline configurations to Settings API (DT docs) — *"Routing management is restricted to administrators (`settings:objects:admin`). Administrators can grant write access to a configuration scope routing via policies."*, OpenPipeline API (DT docs) — *"The Configurations API is deprecated and reached its end of life on June 29, 2026."*

1. OpenPipeline Rule Migration

OpenPipeline processing rules define how ingested data is enriched, extracted, routed, and masked before it lands in Grail. These rules must be migrated **after** Grail buckets exist in the target tenant (because routing rules reference bucket names) and **before** SLOs and alerts (because they consume processed data).

Processing Rule Types

Rule Type	What It Does	Migration Notes
Enrichment	Add fields from lookup tables or entity properties	Port as-is if field names match
Extraction	Parse structured data from unstructured content (DPL patterns)	Port as-is
Routing	Direct data to specific Grail buckets based on conditions	**Update bucket references** to match target bucket names
Masking	Redact sensitive data (PII, credentials, tokens)	Port as-is — verify compliance requirements match

Enrichment Propagation Delay

> **Warning:** On Kubernetes clusters (especially EKS), enrichment rules typically take ~45 minutes to propagate cluster-wide after deployment (observed; actual time varies by cluster size and ActiveGate configuration). Routing rules that depend on enriched attributes (e.g., a `primary_tags.*` field or a Kubernetes label you route on) will not match incoming data until propagation completes — data lands in `default_logs` during the gap.
>
> **Mitigation:** If deploying via Terraform or CI/CD, add a validation gate between enrichment deployment and routing deployment. Either use a fixed wait (45-60 min on EKS) or query for enriched fields before proceeding:
> `dql > fetch logs, from:-5m | filter isNotNull(k8s.namespace.name) | summarize count() >`
> Replace `k8s.namespace.name` (the built-in Kubernetes enrichment) with the enriched

> field your routing rules actually match on. If count > 0, enrichment is propagated and routing rules can be deployed safely.

Deployment Order (Critical)

OpenPipeline and related resources must be deployed in strict dependency order:

Order	Resource	Why This Order
1	Grail buckets	Routing rules reference buckets — they must exist first
2	K8s enrichment rules	Tags feed OpenPipeline conditions and downstream queries
—	*Wait for enrichment propagation (~45 min on K8s)*	*Routing rules fail silently if enriched fields are not yet available*
3	OpenPipeline processing rules	Now routing targets and enrichment sources exist
4	Segments	Segment definitions may reference buckets and tags

> **Monaco note:** `monaco deploy` resolves these dependencies automatically within a single run. However, it does **not** wait for enrichment propagation — if you deploy buckets, enrichment, and routing in one `monaco deploy`, routing rules may not match incoming data for ~45 minutes. For production migrations, consider splitting the deploy into two runs with a validation gate between them.

Coordinated Deployment Strategy

For consolidation scenarios where multiple source tenants have different OpenPipeline rules:

Approach	When to Use	Risk
Merge	Rules from both tenants are complementary (different data sources)	Low — rules do not conflict

Approach	When to Use	Risk
Prioritize	Rules overlap — choose the more mature or comprehensive set	Medium — requires rule-by-rule comparison
Redesign	Rules from both tenants are outdated or inefficient	High effort, best long-term result

Export and Import with Monaco

OpenPipeline configuration is Settings 2.0 objects, so export it by schema with `--settings-schema`. Monaco 2.31.0 (10/23/2026) removed the deprecated `openpipeline` type — the release notes: *"Removed support for deprecated OpenPipeline configurations (type `openpipeline`)"* — and its download command no longer has `--only-openpipeline`.

```
# Export OpenPipeline configuration from the source tenant (logs shown).
# Add every builtin:openpipeline.* schema the tenant uses (for example
.pipeline-groups,
# or the spans / bizevents equivalents). The audit query below lists those
changed in the last 30 days.
# --admin-access includes pipelines and ingest sources owned by other users.
monaco download --manifest manifest.yaml --environment source-tenant \
  --settings-schema
builtin:openpipeline.logs.pipelines,builtin:openpipeline.logs.ingest-
sources,builtin:openpipeline.logs.routing \
  --admin-access

# Review and update bucket references in the exported YAML
# Then deploy to target tenant
monaco deploy manifest.yaml --environment target-tenant --dry-run
monaco deploy manifest.yaml --environment target-tenant
```

> **Ownership matters on export.** Pipelines and ingest sources have owners. `--admin-access` downloads *"openpipeline configurations of all owners, not just those owned by the current user"*, so an export without it can miss pipelines other users own. It *"Requires the settings:objects:admin permission"*. Routing is a single object per configuration scope, and deploying it overwrites the target's routing for that scope. In a consolidation, merge the source tenants' routes into one routing table and deploy it once.

> **Sources:** Monaco commands (DT docs), Monaco v2.31.0 release notes (Dynatrace GitHub), Migrate OpenPipeline configurations to Settings API (DT docs) — *"a single object for the configuration scope routing"*, `openpipeline_v2_logs_routing` (Dynatrace GitHub) — *"Deploying an OpenPipeline routing configuration will overwrite the existing one of the same kind"*.
>
> **Warning:** If routing rules reference buckets that do not exist in the target tenant, the deploy will succeed but data will be routed to the `default_logs` bucket instead. Always verify bucket existence first.

Audit Source OpenPipeline Configuration

Query the source tenant to understand which OpenPipeline configurations are actively managed:

```
// Audit log: OpenPipeline configuration changes (last 30 days)
// Data object corrected 09/24/2026. The Dynatrace audit trail is NOT in
`logs`: the former
// `fetch logs | filter matchesPhrase(log.source, "audit")` matched nothing,
or matched an
// unrelated file-based audit log (a database .aud file on the validation
tenant). Environment
// audit records are structured events in `dt.system.events` with event.kind
== "AUDIT_EVENT".
// OpenPipeline configuration is stored as builtin:openpipeline.* settings
schemas.
fetch dt.system.events, from:-30d
| filter event.kind == "AUDIT_EVENT" and event.provider == "SETTINGS"
| filter startsWith(details.dt.settings.schema_id, "builtin:openpipeline.")
| filter in(event.type, {"CREATE", "UPDATE", "DELETE"})
| summarize changes = count(), by:{details.dt.settings.schema_id, user.id}
| sort changes desc
| limit 10
```

Verify OpenPipeline Rules in Target Tenant

After deploying OpenPipeline rules, verify that data is being processed correctly by checking for enrichment fields on recent log records:

```
// Verify OpenPipeline enrichment is active in target tenant
// Check for enriched fields on recent logs
fetch logs, from:-1h
| filter isNotNull(dt.security_context)
| summarize enriched_records = count()
| fieldsAdd status = if(enriched_records &gt; 0, then: "Enrichment rules
active", else: "No enriched records — check OpenPipeline config")
```

2. Grail Bucket Configuration

Grail buckets control data storage, retention, and access. For consolidation scenarios, bucket naming is critical — multiple source tenants may have identically named custom buckets.

Bucket Naming Strategy for Consolidation

Strategy	Pattern	Example	Best For
Prefix by source	`<source>_<original>`	`us-east_app_logs`	Initial migration — avoids all collisions
Prefix by business unit	`<bu>_<original>`	`retail_app_logs`	Post-merger with clear org boundaries
Unified	`<original>` with enrichment tags	`app_logs` + `source_tenant` tag	Long-term target after validation

> **Recommendation:** Start with prefixed buckets. You can consolidate into unified buckets later once data flows are validated. Starting with separate buckets preserves the ability to roll back.

Retention Alignment

When consolidating tenants, retention policies may differ. Align on the **longer** retention period to avoid compliance violations:

Source Tenant	Source Retention	Target Retention	Rationale
US-East	90 days	360 days	EU-West has PCI requirement — use longer
EU-West	360 days	360 days	Compliance driver
Security logs	365 days	365 days	Regulatory minimum
Dev/staging	7 days	14 days	Increase slightly for migration debugging

Inventory Target Tenant Buckets

After bucket deployment, verify all expected buckets exist with correct retention:

```
// List all Grail buckets with retention and estimated size
fetch dt.system.buckets
| fields display_name, dt.system.table, retention_days,
estimated_uncompressed_bytes
| fieldsAdd size_gb = round(estimated_uncompressed_bytes / 1073741824.0,
decimals: 2)
| sort size_gb desc
```

Verify Data Is Landing in Correct Buckets

After OpenPipeline routing rules are deployed, confirm that data is being routed to the intended buckets:

```
// Verify log routing: count records per bucket (last 1 hour)
fetch logs, from:-1h
| summarize record_count = count(), by:{dt.system.bucket}
| sort record_count desc
```

3. SLO Migration

SLOs exist in two generations, and a SaaS-to-SaaS move is the natural point to land them on the right one:

	Modern SLO app (target default)	SLO Classic
SLI	DQL query producing an `sli` field	Metric-selector expression + entity selector
Monaco	`type: slo-v2` (Monaco v2.22+)	classic SLO API / `builtin:monitoring.slo`
Terraform	`dynatrace_platform_slo` (OAuth client)	`dynatrace_slo_v2` (classic API token)

The two names that look alike are not the same thing: Monaco's `slo-v2` type is the **modern** Grail-based SLO, while the Terraform resource `dynatrace_slo_v2` is **classic**. Earlier versions of this step paired them as if they were one path. Classic SLOs in the source tenant should be **rewritten** onto the modern app in the target rather than copied — the readiness scan flags the classic surfaces as blocked at upgrade, and Dynatrace's own upgrade guidance expects a manual review of each SLO (SLO-05 §2). Whichever generation you are moving, hard-coded entity IDs inside the SLI are **not portable** and must be audited before migration.

SLO Migration Workflow

Step	Action	Tool
1	Export SLO definitions from source tenant	Modern: `monaco download --only-slo-v2` · Classic: `terraform-provider-dynatrace -export dynatrace_slo_v2` (inventory for the rewrite)
2	Audit SLIs for entity IDs	Manual review or script
3	Replace entity IDs with tags or segments; rewrite classic metric expressions as DQL SLIs	Manual edit
4	Deploy SLOs to target tenant	`monaco deploy` or `terraform apply` (`dynatrace_platform_slo`)
5	Verify SLO evaluation after data accumulates	DQL query (below)

Entity ID to Tag Conversion

On the modern app, scope the SLI with a **segment** rather than an entity ID. The segment is itself migrated configuration (`dynatrace_segment`, or `type: segment` in Monaco), so the reference resolves in the target tenant where a `SERVICE-…` ID would not.

```
# BEFORE — DQL SLI pinned to a source-tenant entity ID (breaks on migration)
| filter dt.entity.service == "SERVICE-5E6F7A8B"

# AFTER — no ID in the query; scope comes from custom_sli.filter_segments,
# pointing at a segment that is deployed to the target tenant first
```

Classic — only while a classic SLO is still being inventoried in the source:

```
# BEFORE (hardcoded entity ID — breaks on migration)
metric_expression = "(100)*
(builtin:service.errors.server.successCount:splitBy()):merge(0):default(0):sor
t(value(auto,descending)):limit(20):names:fold(auto)/((builtin:service.request
Count.server:splitBy()):merge(0):default(0):sort(value(auto,descending)):limit
(20):names:fold(auto))"
entity_filter = "type(SERVICE),entityId(SERVICE-5E6F7A8B)"

# AFTER (tag-based — survives migration)
entity_filter = "type(SERVICE),tag(app:checkout),tag(env:production)"
```

Evaluation Window Considerations

SLOs with rolling evaluation windows (e.g., 30-day) will show incomplete data in the target tenant until sufficient history accumulates:

Evaluation Window	Time to Full Data	Recommendation
7-day rolling	7 days	Accept partial data during parallel period
30-day rolling	30 days	Consider shortening to 7-day temporarily
Calendar month	End of month	Start SLOs at beginning of calendar month

> **Important:** SLOs that report 0% or 100% during the transition are expected. Communicate this to stakeholders before enabling SLOs in the target tenant.

Terraform SLO Resource Pattern

```
# Modern SLO app — OAuth client (DT_CLIENT_ID / DT_CLIENT_SECRET /
DT_ACCOUNT_ID, slo:slos:*)
resource "dynatrace_platform_slo" "checkout_availability" {
  name        = "Checkout Service Availability"
  description = "Migrated from the source tenant's classic SLO; SLI rewritten
as DQL"
  tags        = ["app:checkout", "env:production"]

  criteria {
    criteria_detail {
      target         = 99.9
      warning        = 99.95
      timeframe_from = "now-7d" # shortened during the fill period — see the
table above
      timeframe_to   = "now"
    }
  }

  custom_sli {
    indicator = &lt;&lt;-EOT
      timeseries {
        total    = sum(dt.service.request.count),
        failures = sum(dt.service.request.failure_count)
      }
      | fieldsAdd sli = ((total[] - failures[]) / total[]) * 100
      | fieldsRemove total, failures
    EOT

    # Scope comes from a segment deployed to the target tenant, not an entity
ID
    filter_segments {
      filter_segment {
        id = dynatrace_segment.checkout_production.id
      }
    }
  }
}
```

The SLI is SLO-02's availability query; resource shape and scoping options are in SLO-05 and AUTOM-04 §4. `dynatrace_platform_slo` is **excluded from a default Terraform export** — name it explicitly (`-export dynatrace_platform_slo`) when you inventory the target afterwards.

Classic — `dynatrace_slo_v2`, only if the target tenant is still on the classic SLO surface:

```
resource "dynatrace_slo_v2" "checkout_availability" {
  name               = "Checkout Service Availability"
  enabled            = true
  evaluation_type    = "AGGREGATE"
  evaluation_window  = "-1w"
  target_success     = 99.9
  target_warning     = 99.95
  metric_expression  = "(100)*
(builtin:service.errors.server.successCount:splitBy()):merge(0):default(0)/((b
uiltin:service.requestCount.server:splitBy()):merge(0):default(0))"
  filter             = "type(SERVICE),tag(app:checkout),tag(env:production)"

  # required block — omitting it fails terraform apply schema validation
  error_budget_burn_rate {
    burn_rate_visualization_enabled = true
  }
}
```

Verify SLO Evaluation in Target Tenant

After SLOs are deployed and data has accumulated, verify that they are evaluating correctly:

```
// Audit log: SLO configuration changes (last 7 days)
// Use this to verify SLOs were successfully deployed to target tenant
// Data object corrected 09/24/2026. The Dynatrace audit trail is NOT in
`logs`: the former
// `fetch logs | filter matchesPhrase(log.source, "audit")` matched nothing,
or matched an
// unrelated file-based audit log (a database .aud file on the validation
tenant). Environment
// audit records are structured events in `dt.system.events` with event.kind
== "AUDIT_EVENT".
// SLO service writes are provider "SLO" (evaluation calls excluded); classic
SLOs are the
// builtin:monitoring.slo settings schema.
fetch dt.system.events, from:-7d
| filter event.kind == "AUDIT_EVENT"
| filter (event.provider == "SLO" and in(event.type, {"POST", "PUT",
"DELETE"}) and not endsWith(resource, "evaluation:start"))
    or (event.provider == "SETTINGS" and details.dt.settings.schema_id ==
"builtin:monitoring.slo")
| summarize changes = count(), by:{event.provider, event.type, user.id}
| sort changes desc
| limit 10
```

4. Alerting and Notification Migration

Alerting configuration includes three components that must be migrated together: alerting profiles (which problems trigger alerts), notification rules (where alerts are sent), and maintenance windows (when alerts are suppressed).

Alerting Migration Sequence

Order	Component	Dependencies	Monaco Type
1	Alerting profiles	None	`settings` (builtin:alerting.profile)
2	Notification integrations	Webhook URLs, credential vault entries	`settings` (builtin:problem.notifications)
3	Notification rules	Alerting profiles must exist	`settings`

Order	Component	Dependencies	Monaco Type
4	Maintenance windows	Management zones (for scope)	`settings` (builtin:alerting.maintenance-window)

Webhook URL Updates

Notification rules that send alerts to external systems (PagerDuty, Slack, ServiceNow, email) typically use tenant-specific webhook URLs or API keys:

Integration	What Changes	Action
PagerDuty	Integration key (routing key)	May keep same key if routing to same PD service
Slack	Webhook URL	Update if using per-tenant Slack app
ServiceNow	Instance URL, credentials	Update credential vault entry in target
Email	Recipient list	Review and update distribution lists
Custom webhook	Endpoint URL, authentication	Update URL and credentials

> **Important during parallel operation:** agent data does not produce duplicates — each host reports to one tenant. Duplicates come from what you deliberately run in **both** tenants: synthetic monitors, cloud connections, and dual-fed log or metric ingest. Disable each of those in the source as its target counterpart is validated. Do not disable alerting in the target; route alerts for recently migrated hosts to a staging channel while their baselines relearn (**FAQ-25** § 5).

Maintenance Window Strategy

Approach	When to Use
Staging channel (recommended)	Route alerts for recently migrated hosts to a staging channel; keep detection running so you can measure the noise
Wave-scoped maintenance window	Cover only the hosts in a wave, only for its cutover window

Approach	When to Use
Tenant-wide maintenance window	Avoid — it suppresses real problems on every migrated host for the whole overlap

5. Data Retention Optimization

The migration is an opportunity to rationalize data retention across data types. In the source tenant, retention policies may have accumulated organically. The target tenant starts clean.

Recommended Retention by Data Type

Data Type	Production	Staging	Security/Compliance
Logs	35–90 days	7–14 days	365 days
Spans	35 days	7 days	90 days
Metrics	15 months included (extend up to 10 years)	15 months	As compliance requires, up to 10 years
Events	35 days	14 days	90 days
Business events	90–365 days	35 days	365 days

Cost Optimization Strategies

Strategy	Impact	Implementation
Reduce verbose log retention	High	Route debug/info logs to short-retention bucket
Drop noisy spans	Medium	OpenPipeline filter for health-check spans
Aggregate before store	Medium	Use OpenPipeline extraction to create metrics from logs

Strategy	Impact	Implementation
Separate compliance data	Low effort, high governance value	Dedicated bucket with extended retention

Compare Source vs Target Bucket Sizes

Run this query in both tenants to compare data volumes and confirm parity:

```
// Bucket size comparison — run in both source and target tenants
fetch dt.system.buckets
| fields display_name, dt.system.table, retention_days,
estimated_uncompressed_bytes
| fieldsAdd size_gb = round(estimated_uncompressed_bytes / 1073741824.0,
decimals: 2)
| fieldsAdd cost_tier = if(retention_days &gt; 365, then: "Extended",
    else: if(retention_days &gt; 90, then: "Standard", else: "Short"))
| sort size_gb desc
```

6. Step Completion Checklist

Before proceeding to **Step 8 — Enable**, confirm that you have completed each item:

Checkpoint	Status
Grail buckets created in target tenant with correct retention	[ ]
OpenPipeline rules deployed (enrichment, extraction, routing, masking)	[ ]
Data is landing in correct buckets (verified via DQL)	[ ]
Enrichment fields are present on new records	[ ]
SLO definitions exported, entity IDs replaced with tags or segments, classic SLIs rewritten as DQL	[ ]
SLOs deployed to target tenant	[ ]
Alerting profiles migrated	[ ]
Notification rules migrated with updated webhook URLs	[ ]

Checkpoint	Status
Alert routing for the parallel period decided (staging channel or wave-scoped maintenance windows)	[ ]
Data retention policies aligned and documented	[ ]
Stakeholders informed of SLO evaluation timeline (rolling window fill period)	[ ]

Next Step

> **S2S-08: Step 8 — Enable** — Manage parallel operation, communicate to stakeholders, establish Dynatrace Intelligence baselines, and prepare for cutover.

Completed	Next	Remaining
~~1. Discover~~ → ~~2. Strategize~~ → ~~3. Design~~ → ~~4. Prepare~~ → ~~5. Execute~~ → ~~6. Integrate~~ → ~~7. Expand~~	**8. Enable**	9. Optimize

---

Summary

In Step 7, you:

- Migrated OpenPipeline processing rules (enrichment, extraction, routing, masking) following the dependency order: buckets → enrichment → pipelines → segments
- Configured Grail buckets in the target tenant with a prefix strategy for consolidation and aligned retention policies
- Migrated SLO definitions onto the modern SLO app after replacing hardcoded entity IDs with tags or segments
- Migrated alerting profiles, notification rules, and maintenance windows with updated webhook URLs
- Optimized data retention across data types for cost and compliance

Additional Resources

- OpenPipeline Documentation
- Organize your data stored in Grail (DT docs)
- Service-level objectives (DT docs)
- Alerting and Notifications
- Monaco Configuration as Code

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.